Technical Companion to

Cost-Sensitive Neighborhood Aggregation for Heterophilous Graphs

A Self-Contained Guide for CS Undergraduates

Companion to the paper by Eyal Weiss

Abstract

This document explains every equation and technical concept in the CSNA paper in detail, assuming only undergraduate-level linear algebra, probability, optimization, and graph theory. No prior knowledge of graph neural networks, message passing, or GNN notation is required. Each equation is accompanied by (1) a plain-English description of what it computes, (2) a symbol-by-symbol breakdown, (3) an intuitive explanation of why the computation makes sense, and (4) a concrete numerical example where helpful.

Contents

1 Background: Graphs, Features, and the Classification Problem **2**
1.1 What Is a Graph in This Context? . . . 2
1.2 Edge Homophily Ratio $\mathcal{H}$ . . . 2
1.3 Message Passing (Equation 1) . . . 3
1.4 Why Homophily Makes Aggregation Work—and Heterophily Breaks It . . . 4

2 The CSNA Method: Step by Step **5**
2.1 Step 1: The Learned Projection $\mathbf{W}_g$ . . . 5
2.2 Step 2: Edge Cost — Observed Divergence g_{ij} (Equation 3) . . . 5
2.3 Step 3: The Sigmoid Concordance Score s_{ij} (Equation 4) . . . 6
2.4 Softmax Normalization of Weights . . . 7
2.5 Step 4: Dual-Channel Aggregation (Equations 5–6) . . . 8
2.6 Step 5: The Gating Mechanism (Equation 7) . . . 8
2.7 The Extended Variant: $f = g + h$ (Equation 8) . . . 9
2.8 Calibration Loss (Equation 9) . . . 10
2.9 Architecture Choices . . . 11

3 Theoretical Analysis: When Does Cost-Sensitive Aggregation Help? **12**
3.1 The Contextual Stochastic Block Model (CSBM) . . . 12
3.2 Theorem 1: Signal Degradation Under Mean Aggregation . . . 13
3.3 Theorem 2: Signal Preservation Under Cost-Sensitive Aggregation . . . 14
3.4 Connecting the Theory to CSNA . . . 16

4 Summary of Notation **16**

1 Background: Graphs, Features, and the Classification Problem

1.1 What Is a Graph in This Context?

A **graph** $\mathcal{G} = (V, E)$ consists of:

- A set of **nodes** V with $n = |V|$ nodes, numbered $1, 2, \ldots, n$. Each node represents an entity—a webpage, a Wikipedia article, an actor, etc.
- A set of **edges** $E \subseteq V \times V$. An edge $(i, j) \in E$ means nodes i and j are connected (e.g., there is a hyperlink between two webpages). The graph is *undirected*: if $(i, j) \in E$, then $(j, i) \in E$.

Every node i carries two pieces of information:

1. A **feature vector** $\mathbf{x}_i \in \mathbb{R}^d$. This is a list of d real numbers that describe the node. For webpages, the features might encode word frequencies; for actors, they might encode genre co-occurrences. We stack all feature vectors into a **feature matrix** $\mathbf{X} \in \mathbb{R}^{n \times d}$, where row i is $\mathbf{x}_i$.
2. A **label** $y_i \in \{1, 2, \ldots, C\}$. This is the class or category of the node. The goal of **node classification** is to predict the label of each node using its features and the graph structure. During training, we know the labels of some nodes; at test time, we must predict the labels of the rest.

Numerical Example

Suppose we have 4 nodes (webpages) with 3-dimensional features and 2 classes ("student page" vs. "faculty page"):

Node	Features $\mathbf{x}_i$	Label y_i	Neighbors
1	$[1, 0, 3]$	1 (student)	$\{2, 3\}$
2	$[1, 0, 1]$	1 (student)	$\{1, 4\}$
3	$[0, 5, 2]$	2 (faculty)	$\{1, 4\}$
4	$[0, 4, 1]$	2 (faculty)	$\{2, 3\}$

Here $n = 4$, $d = 3$, $C = 2$. Nodes 1 and 2 (students) are connected to each other, but they are also connected to faculty nodes—this graph has significant heterophily.

1.2 Edge Homophily Ratio $\mathcal{H}$

Key Idea

Homophily measures how often connected nodes share the same label. High homophily means "birds of a feather flock together"; low homophily (heterophily) means connected nodes tend to differ.

The **edge homophily ratio** is defined as:

$$\mathcal{H} = \frac{|\{(i, j) \in E : y_i = y_j\}|}{|E|}$$

What it computes: Count the number of edges whose endpoints share the same label, and divide by the total number of edges.

Symbol breakdown:

- $\{(i,j) \in E : y_i = y_j\}$ — the set of edges where both endpoints have the same label.
- $|\cdot|$ denotes set cardinality (the number of elements).
- $|E|$ is the total number of edges.

Range and interpretation:

- $\mathcal{H} = 1$: every edge connects same-label nodes (**perfect homophily**).
- $\mathcal{H} = 0$: every edge connects different-label nodes (**perfect heterophily**).
- $\mathcal{H} \approx 0.5$: edges are roughly random with respect to labels.
- $\mathcal{H} < 0.5$: the graph is **heterophilous**—connected nodes tend to differ in label.

Numerical Example

In the 4-node graph above, the edges are $\{(1,2),(1,3),(2,4),(3,4)\}$.

- Edge $(1,2)$: both students $\rightarrow$ same label.
- Edge $(1,3)$: student–faculty $\rightarrow$ different label.
- Edge $(2,4)$: student–faculty $\rightarrow$ different label.
- Edge $(3,4)$: both faculty $\rightarrow$ same label.

Same-label edges: 2 out of 4 total, so $\mathcal{H} = 2/4 = 0.5$.
Now suppose we remove edge $(1,2)$ and add edge $(1,4)$ instead. Then all 4 edges are cross-class, giving $\mathcal{H} = 0/4 = 0$—perfect heterophily.
The datasets in the paper have $\mathcal{H}$ between 0.09 and 0.23, meaning that fewer than 1 in 4 edges connect same-class nodes.

1.3 Message Passing (Equation 1)

Message passing is the fundamental operation of Graph Neural Networks (GNNs). The idea is simple: to update node i's representation, *collect information from its neighbors, combine it, and transform the result.*

The paper's Equation 1 defines one round of message passing:

$$\mathbf{h}_i^{(\ell+1)} = \sigma\Big(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}\, \mathbf{W}^{(\ell)}\, \mathbf{h}_j^{(\ell)}\Big) \tag{Eq. 1}$$

What it computes in plain English: To get node i's new representation at layer $\ell+1$, do the following: for each neighbor j (including i itself), transform j's current representation by the weight matrix $\mathbf{W}$, scale it by a weight α_{ij}, sum all these contributions, and pass the result through a nonlinear activation function.

Symbol-by-symbol breakdown:

- $\mathbf{h}_j^{(\ell)} \in \mathbb{R}^d$: the **representation** (also called "embedding" or "hidden state") of node j at layer ℓ. At layer 0, $\mathbf{h}_j^{(0)} = \mathbf{x}_j$ (the raw features). After each layer, representations are updated.
- $\mathbf{W}^{(\ell)} \in \mathbb{R}^{d' \times d}$: a **learnable weight matrix** at layer ℓ. This is the neural network's set of parameters—values that are adjusted during training to minimize classification error. It transforms a d-dimensional vector into a d'-dimensional vector. Multiplication $\mathbf{W}^{(\ell)}\mathbf{h}_j^{(\ell)}$ is a standard matrix–vector product, producing a vector in $\mathbb{R}^{d'}$.
- $\mathcal{N}(i)$: the **neighborhood** of node i, including i itself (via a self-loop). If node i has edges to nodes 2, 5, and 7, then $\mathcal{N}(i) = \{i, 2, 5, 7\}$.

- $\alpha_{ij} \in \mathbb{R}$: the **aggregation weight** for the message from j to i. This controls how much influence neighbor j has on node i's updated representation. In the simplest GNN (GCN), $\alpha_{ij} = 1/\sqrt{d_i \cdot d_j}$ where d_i is the degree of node i—essentially an average. More sophisticated GNNs learn these weights (like attention in GAT).
- $\sum_{j \in \mathcal{N}(i)}$: the **aggregation step**—sum over all neighbors (including self).
- $\sigma(\cdot)$: a **nonlinear activation function**, such as ReLU ($\sigma(x) = \max(0, x)$, applied element-wise). Without nonlinearity, stacking multiple layers would collapse to a single linear transformation.

Numerical Example

Suppose node 1 has neighbors $\mathcal{N}(1) = \{1, 2, 3\}$ with representations:

$$\mathbf{h}_1^{(0)} = [1, 0], \quad \mathbf{h}_2^{(0)} = [0, 2], \quad \mathbf{h}_3^{(0)} = [1, 1]$$

and the weight matrix is $\mathbf{W} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ (identity, for simplicity), with uniform weights $\alpha_{ij} = 1/3$ for all j.
Then:

$$\begin{aligned} \mathbf{h}_1^{(1)} &= \sigma\Big(\tfrac{1}{3}\mathbf{W}[1,0]^\top + \tfrac{1}{3}\mathbf{W}[0,2]^\top + \tfrac{1}{3}\mathbf{W}[1,1]^\top\Big) \\ &= \sigma\big(\tfrac{1}{3}[1,0] + \tfrac{1}{3}[0,2] + \tfrac{1}{3}[1,1]\big) \\ &= \sigma([2/3,\ 1]) = [2/3,\ 1] \quad \text{(using ReLU)} \end{aligned}$$

The new representation of node 1 is the average of its neighbors' (transformed) features, passed through ReLU.

1.4 Why Homophily Makes Aggregation Work—and Heterophily Breaks It

Key Idea

If your neighbors mostly share your label, averaging their features reinforces your class identity. If your neighbors mostly have a different label, averaging their features *dilutes* or *overwrites* your class identity.

Imagine you are a student node with features that "look like a student." Your neighbors are:

Homophilous case ($\mathcal{H}$ high): Most neighbors are also students. Averaging their features produces something that still looks like a student—possibly an even cleaner signal because noise averages out. After aggregation, student nodes become *more* similar to each other and *more* distinct from faculty nodes. Classification is easy.

Heterophilous case ($\mathcal{H}$ low): Most neighbors are faculty. Averaging their features produces something that looks like a mix of student and faculty—or even more like faculty than student. After aggregation, your representation has been *pulled toward* the other class. Student and faculty representations become harder to distinguish. Classification suffers.

This is why, on highly heterophilous graphs (like Texas with $\mathcal{H} = 0.09$), a simple Multi-Layer Perceptron (MLP) that *ignores the graph entirely* often outperforms standard GNNs: it is better to use only your own features than to average in features from mostly-wrong-class neighbors.

The CSNA paper's central question is: *Can we design smarter aggregation weights that avoid*

this dilution?

2 The CSNA Method: Step by Step

CSNA (Cost-Sensitive Neighborhood Aggregation) modifies message passing by assigning each edge an individualized weight based on how similar the two endpoints look in a learned projection space. We now walk through each step.

2.1 Step 1: The Learned Projection $\mathbf{W}_g$

Before measuring how similar two nodes are, CSNA first *projects* their representations into a new space using a learned matrix $\mathbf{W}_g$.

Why not just compare the raw representations? Raw features live in a high-dimensional space (e.g., $d = 1703$ for the webpage datasets) where many dimensions may be irrelevant to class identity. Two students might differ wildly in raw features (one's page discusses algorithms, another discusses databases), but in a well-chosen lower-dimensional projection, the class-relevant signal is amplified and the irrelevant variation is discarded.

This is analogous to dimensionality reduction techniques like PCA, but here the projection $\mathbf{W}_g \in \mathbb{R}^{d' \times d}$ is *learned end-to-end* via backpropagation—the neural network discovers which projection makes same-class nodes look similar and different-class nodes look different.

2.2 Step 2: Edge Cost — Observed Divergence g_{ij} (Equation 3)

$$g_{ij}^{(\ell)} = \|\mathbf{W}_g\, \mathbf{h}_i^{(\ell)} - \mathbf{W}_g\, \mathbf{h}_j^{(\ell)}\|_2 \tag{Eq. 3}$$

What it computes: The Euclidean (L2) distance between node i and node j after both are projected into the learned space.

Symbol breakdown:

- $\mathbf{h}_i^{(\ell)} \in \mathbb{R}^d$: node i's representation at layer ℓ.
- $\mathbf{W}_g \in \mathbb{R}^{d' \times d}$: the learned projection matrix. d' is typically smaller than d (e.g., 64 vs. 1703).
- $\mathbf{W}_g \mathbf{h}_i^{(\ell)} \in \mathbb{R}^{d'}$: node i's projected representation.
- $\mathbf{W}_g \mathbf{h}_i^{(\ell)} - \mathbf{W}_g \mathbf{h}_j^{(\ell)} \in \mathbb{R}^{d'}$: the difference vector between the two projected representations.
- $\|\cdot\|_2$: the L2 (Euclidean) norm, i.e., $\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_{d'}^2}$.
- $g_{ij}^{(\ell)} \geq 0$: a non-negative scalar. Low g_{ij} means "these nodes look similar in the projection" (likely same class); high g_{ij} means "these nodes look different" (likely different class).

Why "observed divergence"? The paper calls g_{ij} the "observed divergence" because it is computed directly from the observed (current) representations—no additional learned scoring function is needed. It is a direct measurement of how far apart i and j appear to be.

Numerical Example

Suppose $d' = 2$ and the projected representations are:

$$\mathbf{W}_g \mathbf{h}_1 = [3, 1], \quad \mathbf{W}_g \mathbf{h}_2 = [3, 0], \quad \mathbf{W}_g \mathbf{h}_3 = [0, 4]$$

Then:

$$g_{12} = \|[3,1] - [3,0]\|_2 = \|[0,1]\|_2 = 1 \quad \text{(small: similar)}$$
$$g_{13} = \|[3,1] - [0,4]\|_2 = \|[3,-3]\|_2 = \sqrt{9+9} = \sqrt{18} \approx 4.24 \quad \text{(large: different)}$$

If nodes 1 and 2 are same-class and node 3 is a different class, the projection has successfully made same-class nodes close and different-class nodes far apart.

2.3 Step 3: The Sigmoid Concordance Score s_{ij} (Equation 4)

$$s_{ij}^{(\ell)} = \sigma\left(\frac{-g_{ij}^{(\ell)}}{\tau}\right) \in (0,1) \tag{Eq. 4}$$

What it computes: Converts the distance g_{ij} into a "concordance score" between 0 and 1. High score ($s_{ij} \approx 1$) means the edge is likely **concordant** (same class); low score ($s_{ij} \approx 0$) means it is likely **discordant** (different class).

Symbol breakdown:

- $g_{ij}^{(\ell)} \geq 0$: the distance from Eq. 3.
- $\tau > 0$: a **temperature** hyperparameter that controls the sharpness of the sigmoid. Tuned from $\{0.1, 0.5, 1.0, 2.0\}$ in experiments.
- $-g_{ij}/\tau$: a non-positive number. The minus sign means that *larger distance → more negative input → lower sigmoid output.*
- $\sigma(z) = 1/(1+e^{-z})$: the **sigmoid function**, which maps any real number to $(0,1)$.

Why the sigmoid? The sigmoid provides a smooth, differentiable mapping from distance to a score in $(0,1)$, which is needed for two reasons: (1) we need a probabilistic-style weight to split messages between two channels, and (2) the function must be differentiable so gradients can flow back to $\mathbf{W}_g$ during training.

What does temperature τ control?

- Small τ (e.g., 0.1): the sigmoid transition is *sharp*. Even a small distance produces $s_{ij} \approx 0$ (hard routing).
- Large τ (e.g., 2.0): the sigmoid transition is *gradual*. Moderate distances still produce s_{ij} near 0.5 (soft routing).

Think of τ as controlling how "decisive" the routing is.

Numerical Example

Continuing from the previous example with $\tau = 1$:

$$s_{12} = \sigma(-1/1) = \sigma(-1) = \frac{1}{1+e^1} \approx \frac{1}{1+2.72} \approx 0.27$$
$$s_{13} = \sigma(-4.24/1) = \sigma(-4.24) = \frac{1}{1+e^{4.24}} \approx \frac{1}{1+69.4} \approx 0.014$$

Node pair (1,2) gets concordance 0.27 (moderate—could be same class), while pair (1,3) gets concordance 0.014 (very low—likely different class).
With $\tau = 0.1$: $s_{12} = \sigma(-10) \approx 0.00005$ and $s_{13} = \sigma(-42.4) \approx 0$. Even the moderately-close pair gets near-zero concordance—sharp temperature leaves little room for nuance.

With $\tau = 5$: $s_{12} = \sigma(-0.2) \approx 0.45$ and $s_{13} = \sigma(-0.85) \approx 0.30$. Both scores are moderate—soft temperature makes the routing very gentle.
Note: $s_{ii} = \sigma(0/\tau) = \sigma(0) = 0.5$ for self-loops, since $g_{ii} = 0$.

2.4 Softmax Normalization of Weights

Before using the concordance scores as aggregation weights, they are normalized using the **softmax** function, computed *per node*:

$$\tilde{s}_{ij} = \text{softmax}_{j \in \mathcal{N}(i)}(s_{ij}) = \frac{\exp(s_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(s_{ik})}$$

Similarly, the discordance weights use $1 - s_{ij}$:

$$\tilde{d}_{ij} = \text{softmax}_{j \in \mathcal{N}(i)}(1 - s_{ij}) = \frac{\exp(1 - s_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(1 - s_{ik})}$$

What softmax does here: For each node i, softmax converts the raw concordance scores of all its neighbors into a probability distribution that sums to 1. Neighbors with higher concordance get a larger share of the total weight in the concordant channel; neighbors with higher discordance $(1 - s_{ij})$ get a larger share in the discordant channel.

Why normalize per node? Without normalization, a node with many neighbors would accumulate a much larger aggregated signal than a node with few neighbors. Per-node softmax ensures every node's aggregated representation is a weighted *average* (weights summing to 1) of its neighbors, regardless of degree.

Why softmax instead of simple division? Softmax amplifies differences: if one neighbor has concordance 0.9 and another has 0.2, softmax gives the first neighbor a disproportionately large share (because $\exp(0.9) \gg \exp(0.2)$). Simple normalization $(s_{ij}/\sum_k s_{ik})$ would produce more uniform weights.

Numerical Example

Suppose node 1 has three neighbors (including itself) with concordance scores:

$$s_{11} = 0.5, \quad s_{12} = 0.4, \quad s_{13} = 0.05$$

Softmax for the concordant channel:

$$\begin{aligned}
\exp(0.5) \approx 1.649, \quad \exp(0.4) \approx 1.492, \quad \exp(0.05) \approx 1.051 \\
\text{Sum} = 1.649 + 1.492 + 1.051 = 4.192 \\
\tilde{s}_{11} = 1.649/4.192 \approx 0.393 \\
\tilde{s}_{12} = 1.492/4.192 \approx 0.356 \\
\tilde{s}_{13} = 1.051/4.192 \approx 0.251
\end{aligned}$$

Discordant channel uses $1 - s_{ij}$: values are $0.5, 0.6, 0.95$.

$$\begin{aligned}
\exp(0.5) \approx 1.649, \quad \exp(0.6) \approx 1.822, \quad \exp(0.95) \approx 2.586 \\
\text{Sum} = 1.649 + 1.822 + 2.586 = 6.057 \\
\tilde{d}_{11} \approx 0.272, \quad \tilde{d}_{12} \approx 0.301, \quad \tilde{d}_{13} \approx 0.427
\end{aligned}$$

Notice: in the concordant channel, the self-loop and same-class neighbor (node 2) get the most weight. In the discordant channel, the different-class neighbor (node 3) gets the most weight. The channels emphasize different parts of the neighborhood.

2.5 Step 4: Dual-Channel Aggregation (Equations 5–6)

$$\mathbf{h}_i^{\text{con}} = \sum_{j \in \mathcal{N}(i)} \tilde{s}_{ij}\, \mathbf{W}_{\text{con}}\, \mathbf{h}_j^{(\ell)} \tag{Eq. 5}$$

$$\mathbf{h}_i^{\text{dis}} = \sum_{j \in \mathcal{N}(i)} \tilde{d}_{ij}\, \mathbf{W}_{\text{dis}}\, \mathbf{h}_j^{(\ell)} \tag{Eq. 6}$$

What they compute: Two separate aggregated representations for node i:

- $\mathbf{h}_i^{\text{con}}$ (**concordant channel**): a weighted sum of neighbors' features, where likely-same-class neighbors contribute more. Transformed by $\mathbf{W}_{\text{con}}$.
- $\mathbf{h}_i^{\text{dis}}$ (**discordant channel**): a weighted sum where likely-different-class neighbors contribute more. Transformed by a *different* matrix $\mathbf{W}_{\text{dis}}$.

Symbol breakdown:

- $\tilde{s}_{ij}$: the softmax-normalized concordance weight (a scalar between 0 and 1, summing to 1 over j).
- $\tilde{d}_{ij}$: the softmax-normalized discordance weight.
- $\mathbf{W}_{\text{con}} \in \mathbb{R}^{d' \times d}$: weight matrix for the concordant channel.
- $\mathbf{W}_{\text{dis}} \in \mathbb{R}^{d' \times d}$: weight matrix for the discordant channel.
- $\mathbf{h}_j^{(\ell)} \in \mathbb{R}^d$: neighbor j's current representation.
- $\mathbf{h}_i^{\text{con}}, \mathbf{h}_i^{\text{dis}} \in \mathbb{R}^{d'}$: the resulting channel representations for node i.

Why Two DIFFERENT Weight Matrices Matter

This is the heart of CSNA's design. The concordant and discordant channels use *independent* weight matrices $\mathbf{W}_{\text{con}}$ and $\mathbf{W}_{\text{dis}}$.
Why? Because same-class neighbors carry a fundamentally different type of information than different-class neighbors:

- **Same-class neighbors** reinforce your identity. Their features, when transformed by $\mathbf{W}_{\text{con}}$, should produce representations that sharpen your class signal. The concordant transformation might learn to extract and amplify class-specific features.
- **Different-class neighbors** carry complementary (or contradictory) information. Their features, when transformed by $\mathbf{W}_{\text{dis}}$, might be used in a "negative" sense: "if my different-class neighbors look like X, then I am probably not-X." The discordant transformation can learn to extract useful contrast information.

If both channels used the same $\mathbf{W}$, there would be no way to treat the two types of information differently—the model would collapse to standard weighted aggregation.

2.6 Step 5: The Gating Mechanism (Equation 7)

$$\mathbf{h}_i^{(\ell+1)} = \sum_{k \in \{\text{con,dis,self}\}} \gamma_k(\mathbf{h}_i) \cdot \mathbf{h}_i^k \tag{Eq. 7}$$

where:

$$\mathbf{h}_i^{\text{self}} = \mathbf{W}_{\text{self}}\, \mathbf{h}_i^{(\ell)}$$

$$\gamma(\mathbf{h}_i) = \text{softmax}\big(\mathbf{W}_\gamma\, [\mathbf{h}_i^{\text{con}} \| \mathbf{h}_i^{\text{dis}} \| \mathbf{h}_i^{\text{self}}]\big) \in \mathbb{R}^3$$

What it computes: The final updated representation of node i is a weighted combination of three ingredients:

1. $\mathbf{h}_i^{\text{con}}$: what likely-same-class neighbors say (concordant channel).
2. $\mathbf{h}_i^{\text{dis}}$: what likely-different-class neighbors say (discordant channel).
3. $\mathbf{h}_i^{\text{self}}$: what the node itself says (ego channel, ignoring the graph).

The gate weights $\gamma_{\text{con}}, \gamma_{\text{dis}}, \gamma_{\text{self}}$ determine the mixing proportions, and they *depend on the node's own representation*—different nodes can rely on different channels.

Symbol breakdown:

- $\mathbf{W}_{\text{self}} \in \mathbb{R}^{d' \times d}$: weight matrix for the ego (self) channel. This transforms the node's own representation without any neighbor information.
- $[\mathbf{h}_i^{\text{con}} \| \mathbf{h}_i^{\text{dis}} \| \mathbf{h}_i^{\text{self}}] \in \mathbb{R}^{3d'}$: the concatenation of all three channel outputs. If each channel produces a 64-dimensional vector, this is a 192-dimensional vector.
- $\mathbf{W}_\gamma \in \mathbb{R}^{3 \times 3d'}$: a small weight matrix that produces 3 scores (one per channel) from the concatenated representation.
- $\text{softmax}(\cdot) \in \mathbb{R}^3$: converts the 3 raw scores into a probability distribution: $\gamma_{\text{con}} + \gamma_{\text{dis}} + \gamma_{\text{self}} = 1$, with each $\gamma_k \geq 0$.
- The final output is: $\gamma_{\text{con}} \cdot \mathbf{h}_i^{\text{con}} + \gamma_{\text{dis}} \cdot \mathbf{h}_i^{\text{dis}} + \gamma_{\text{self}} \cdot \mathbf{h}_i^{\text{self}}$.

Why per-node gating? Different nodes benefit from different information sources. A node surrounded by mostly-same-class neighbors should rely on the concordant channel. A node with many different-class neighbors that carry useful contrast information should lean toward the discordant channel. A node whose neighbors are uninformative should fall back to its own features (self channel). The per-node gate learns to make this decision for each node individually.

Why softmax over 3 channels? Softmax ensures the gate weights are non-negative and sum to 1, so the output is a convex combination of the three channel representations. This prevents any channel from having negative influence (which could destabilize training) and ensures the output scale stays bounded.

Numerical Example

Suppose for node i the gate logits are $[1.0, -0.5, 2.0]$:

$$\begin{aligned}
\gamma &= \text{softmax}([1.0, -0.5, 2.0]) \\
&= \frac{1}{e^{1.0} + e^{-0.5} + e^{2.0}} [e^{1.0}, e^{-0.5}, e^{2.0}] \\
&\approx \frac{1}{2.72 + 0.61 + 7.39} [2.72, 0.61, 7.39] \\
&\approx [0.254, 0.057, 0.689]
\end{aligned}$$

This node relies 69% on its own features, 25% on the concordant channel, and only 6% on the discordant channel—similar to an MLP with a small graph-based correction.
On the Wisconsin and Actor datasets, the paper reports that CSNA learns $\gamma_{\text{self}} = 1.0$ for all nodes, effectively ignoring the graph entirely and reducing to an MLP.

2.7 The Extended Variant: $f = g + h$ (Equation 8)

$$f_{ij}^{(\ell)} = g_{ij}^{(\ell)} + h_{ij}^{(\ell)}, \quad \text{where} \quad h_{ij}^{(\ell)} = \text{softplus}\big(\mathbf{a}^\top [\mathbf{W}_g \mathbf{h}_i^{(\ell)} \| \mathbf{W}_g \mathbf{h}_j^{(\ell)}]\big) \tag{Eq. 8}$$

What it computes: An enriched edge cost that adds a *learned asymmetric* component h_{ij} on top of the observed divergence g_{ij}. In the default "lite" variant, $f_{ij} = g_{ij}$ (no h component). In the "extended" variant, $f_{ij} = g_{ij} + h_{ij}$ replaces g_{ij} in the concordance formula (Eq. 4).

Symbol breakdown:

- g_{ij}: the observed divergence from Eq. 3 (L2 distance in projection space).
- $[\mathbf{W}_g\mathbf{h}_i \| \mathbf{W}_g\mathbf{h}_j] \in \mathbb{R}^{2d'}$: the **concatenation** of i's and j's projected representations. If each projected representation is 64-dimensional, this is a 128-dimensional vector.
- $\mathbf{a} \in \mathbb{R}^{2d'}$: a learnable parameter vector.
- $\mathbf{a}^\top[\cdot] \in \mathbb{R}$: a dot product, producing a single scalar. This is essentially a learned linear scoring function on the pair (i, j).
- $\text{softplus}(z) = \ln(1 + e^z)$: a smooth, always-positive activation function. It ensures $h_{ij} \geq 0$, so that h can only *increase* the total cost (never decrease it below g_{ij}).
- $f_{ij} = g_{ij} + h_{ij} \geq g_{ij} \geq 0$: the combined cost.

What does h_{ij} add? The observed divergence g_{ij} is symmetric ($g_{ij} = g_{ji}$) and purely distance-based—it only knows *how far apart* two nodes are. The learned component h_{ij} can capture more complex patterns by looking at the actual values of the projected representations (not just their difference). The concatenation $[\mathbf{W}_g\mathbf{h}_i \| \mathbf{W}_g\mathbf{h}_j]$ lets $\mathbf{a}$ learn different roles for the source and target node features.

Why softplus for non-negativity? The cost function should not be able to "cancel out" the observed divergence. If h_{ij} could be negative, it could make two distant nodes appear close (cost near zero), defeating the purpose of measuring divergence. Softplus ensures $h_{ij} > 0$, so the learned component can only add cost, not subtract it.

Practical impact: In the paper's experiments, the extended variant wins clearly only on Wisconsin (84.1% vs. 79.6%) and is comparable or slightly worse on all other datasets. The lite version (just g_{ij}) is the default because the added parameters of h_{ij} risk overfitting on the small datasets in the benchmark suite.

2.8 Calibration Loss (Equation 9)

$$\mathcal{L}_{\text{cal}} = \frac{1}{|E_{\text{train}}|} \sum_{(i,j) \in E_{\text{train}}} \left[\text{ReLU}(g_{ij} - \mathbb{1}[y_i \neq y_j])\right]^2 \qquad \text{(Eq. 9)}$$

What it computes: A penalty term that encourages the cost function g_{ij} to assign *low* cost to same-class edges. It is added to the main classification loss during training.

Symbol breakdown:

- E_{train}: the set of edges where both endpoints have known labels (i.e., both are in the training set).
- $\mathbb{1}[y_i \neq y_j]$: the **indicator function**—equals 1 if i and j have different labels, and 0 if they have the same label.
- $g_{ij} - \mathbb{1}[y_i \neq y_j]$: the "excess cost."
  - For same-class edges ($y_i = y_j$): this equals $g_{ij} - 0 = g_{ij}$. Any positive cost is penalized.
  - For different-class edges ($y_i \neq y_j$): this equals $g_{ij} - 1$. Only cost above 1 is penalized (cost up to 1 is "free").
- $\text{ReLU}(z) = \max(0, z)$: clips negative values to zero. Only *excess* cost incurs a penalty.

- $[\cdot]^2$: squared penalty, so larger violations are penalized more heavily.
- The sum is averaged over the number of training edges.

What it encourages:

- Same-class edges: g_{ij} should be as close to 0 as possible (these nodes should look similar in the projection).
- Different-class edges: g_{ij} can be up to 1 without penalty, and is only penalized if it exceeds 1 (preventing extreme costs that could cause numerical issues).

The Scale Mismatch

The indicator $\mathbb{1}[y_i \neq y_j]$ is either 0 or 1, while g_{ij} (an L2 distance in a learned space) can take any non-negative value—potentially much larger than 1. This means the regularizer acts as a "soft" penalty rather than a precise calibration: it pushes same-class costs toward zero but does not enforce a specific threshold for different-class edges. The paper explicitly notes this scale mismatch.

The full training objective is:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_{\text{cal}} \cdot \mathcal{L}_{\text{cal}}$$

where $\mathcal{L}_{\text{CE}}$ is the standard cross-entropy classification loss and $\lambda_{\text{cal}} = 0.1$ is fixed across all experiments.

Numerical Example

Suppose we have three training edges with costs and labels:

Edge	g_{ij}	Same class?	Penalty contribution
$(1,2)$	0.3	Yes ($\mathbb{1} = 0$)	$\text{ReLU}(0.3-0)^2 = 0.09$
$(1,3)$	2.5	No ($\mathbb{1} = 1$)	$\text{ReLU}(2.5-1)^2 = 2.25$
$(2,3)$	0.8	No ($\mathbb{1} = 1$)	$\text{ReLU}(0.8-1)^2 = \text{ReLU}(-0.2)^2 = 0$

$\mathcal{L}_{\text{cal}} = (0.09 + 2.25 + 0)/3 = 0.78$.
The penalty comes from two sources: edge $(1,2)$ is same-class but has cost 0.3 (should be near 0), and edge $(1,3)$ is different-class but has cost 2.5 (above the "free" threshold of 1). Edge $(2,3)$ is different-class with cost 0.8, which is below 1, so no penalty.

2.9 Architecture Choices

The paper makes several practical design choices that are worth understanding:

Input MLP. Before the first CSNA layer, the raw features $\mathbf{X}$ are passed through a standard Multi-Layer Perceptron (2-layer feedforward network). This projects the possibly very high-dimensional features ($d = 1703$) into a lower-dimensional space, producing the initial representations $\mathbf{h}_i^{(0)}$. The MLP also introduces nonlinearity, which can help separate classes even before any graph information is used.

Residual connections. Between CSNA layers, a **residual connection** adds the input to the output: $\mathbf{h}_i^{(\ell+1)} = \text{CSNA}(\mathbf{h}_i^{(\ell)}) + \mathbf{h}_i^{(\ell)}$. This is the same idea as in ResNets: it makes it easy for the network to learn the identity function ("do nothing") if an additional layer would hurt. It also helps with gradient flow during training.

Gate bias initialization $[0, 0, 1]$**.** The gate bias for $\mathbf{W}_\gamma$ is initialized to $[0, 0, 1]$, which means:

$$\gamma = \text{softmax}([0, 0, 1]) \approx [0.21, 0.21, 0.58]$$

This favors the **self (ego) channel** at initialization. The model starts behaving approximately like an MLP, which is a reasonable baseline on heterophilous graphs. During training, the gate can learn to shift weight toward the concordant or discordant channels if the graph structure is useful.

Starting from MLP-like behavior is a deliberate safety measure: if the graph structure turns out to be harmful (as on some heterophilous datasets), the model does not need to "unlearn" reliance on bad graph information—it was never relying on it in the first place.

3 Theoretical Analysis: When Does Cost-Sensitive Aggregation Help?

The theoretical section of the paper asks: *under what conditions does weighting edges by their cost function improve classification over simple mean aggregation?* To answer this precisely, the analysis uses a synthetic graph model.

3.1 The Contextual Stochastic Block Model (CSBM)

Definition 3.1 (Contextual Stochastic Block Model)**.** A graph $\mathcal{G}$ is drawn from $\text{CSBM}(n, 2, p, q, \mu)$:

- n nodes split into two equal classes: V_+ (class $+1$) and V_- (class -1), each of size $n/2$.
- Edges are drawn independently at random:
  - Probability p for an edge between two same-class nodes.
  - Probability q for an edge between two different-class nodes.
- Node features: $\mathbf{x}_i \sim \mathcal{N}(\boldsymbol{\mu}_{y_i}, \mathbf{I}_d)$.
  - Class $+1$ nodes: features are drawn from a Gaussian centered at $\boldsymbol{\mu}_+ = +\frac{\mu}{2}\mathbf{e}_1$.
  - Class -1 nodes: features are drawn from a Gaussian centered at $\boldsymbol{\mu}_- = -\frac{\mu}{2}\mathbf{e}_1$.

What does this model capture?

- The **graph structure** is controlled by p and q:
  - $p > q$: homophilous graph (same-class nodes more likely to connect).
  - $p < q$: heterophilous graph (different-class nodes more likely to connect).
  - $p = q$: random graph (edges independent of class).
- The **node features** are noisy observations of class identity. Each node's feature vector is a random draw from a Gaussian whose center depends on the class. The parameter μ controls the **signal strength**: large μ means the two classes have very different mean features; small μ means the classes are hard to distinguish from features alone.
- $\mathbf{e}_1 = [1, 0, 0, \ldots, 0]$ is the first standard basis vector. The class means differ only in the first coordinate: $\boldsymbol{\mu}_+ - \boldsymbol{\mu}_- = \mu \cdot \mathbf{e}_1$.
- The homophily ratio in this model is $\mathcal{H} = p/(p + q)$.

Numerical Example

With $n = 100$, $p = 0.1$, $q = 0.4$, $\mu = 2$:

- 50 nodes per class.
- Each node connects to $\sim 0.1 \times 49 \approx 5$ same-class nodes and $\sim 0.4 \times 50 = 20$ different-class nodes.

- Homophily: $\mathcal{H} = 0.1/0.5 = 0.2$—highly heterophilous.
- Features: class $+1$ nodes have features centered at $[+1, 0, \ldots, 0]$; class -1 nodes at $[-1, 0, \ldots, 0]$.
- A typical class $+1$ node might have features $[1.3, -0.5, 0.2, \ldots]$ (noisy version of $[1, 0, 0, \ldots]$).

3.2 Theorem 1: Signal Distortion Under Mean Aggregation

Theorem 3.1 (Signal distortion under mean aggregation)**.** *Let $\mathcal{G} \sim \mathrm{CSBM}(n, 2, p, q, \mu)$ with constant $p, q \in (0, 1)$ and $q > p$ (heterophilous). After one round of symmetrically normalized aggregation with self-loops, the expected class-mean difference satisfies:*

$$\mathbb{E}[\bar{\mathbf{h}}_+^{(1)} - \bar{\mathbf{h}}_-^{(1)}] = \frac{p-q}{p+q}(\boldsymbol{\mu}_+ - \boldsymbol{\mu}_-) + O(1/n) \tag{Thm. 1}$$

What it says in plain English: After mean aggregation, the class-mean difference is scaled by the signed factor $\lambda = (p-q)/(p+q)$. When $q > p$, this factor is *negative*, meaning the class centroids swap sides—their relative direction *reverses*. The magnitude $|\lambda|$ is less than 1 (attenuation) whenever $q \neq p$, and is minimized near $p = q$—not in the extreme heterophily limit. In fact, when $q \gg p$, $|\lambda| \to 1$: strong heterophily distorts the signal (sign flip) but does not collapse it.

Symbol breakdown:

- $\bar{\mathbf{h}}_+^{(1)}$: the mean representation of all class $+1$ nodes after one layer of aggregation. $\bar{\mathbf{h}}_+^{(1)} = \frac{2}{n}\sum_{i \in V_+} \mathbf{h}_i^{(1)}$.
- $\bar{\mathbf{h}}_-^{(1)}$: same for class -1.
- $\bar{\mathbf{h}}_+^{(1)} - \bar{\mathbf{h}}_-^{(1)}$: the vector difference between the two class centroids after aggregation.
- $\boldsymbol{\mu}_+ - \boldsymbol{\mu}_-$: the original class-mean difference *before* aggregation.
- $\frac{p-q}{p+q}$: the **scaling factor**. It is *signed*: negative when $q > p$ (heterophily), positive when $p > q$ (homophily). When negative, the class centroids reverse direction.
- $O(1/n)$: a correction term that absorbs the self-loop contribution (negligible under the dense CSBM where degrees are $\Theta(n)$) and concentration errors.

Intuitive explanation: Under mean aggregation, each node's new representation is an average of its neighbors' features. In a CSBM with $q > p$, a typical node has:

- Fraction $\frac{p}{p+q}$ of same-class neighbors (contributing toward $\boldsymbol{\mu}_+$).
- Fraction $\frac{q}{p+q}$ of different-class neighbors (contributing toward $\boldsymbol{\mu}_-$).

The average representation of a class $+1$ node becomes:

$$\frac{p}{p+q}\boldsymbol{\mu}_+ + \frac{q}{p+q}\boldsymbol{\mu}_- = \frac{p-q}{p+q}\boldsymbol{\mu}_+$$

(using $\boldsymbol{\mu}_- = -\boldsymbol{\mu}_+$ in the signal direction).

The class signal $\boldsymbol{\mu}_+$ has been multiplied by $\frac{p-q}{p+q}$, which is negative (since $q > p$) and has absolute value less than 1. The signal has both *shrunk* and *flipped sign*!

Numerical Example

With $p = 0.1$, $q = 0.4$:

$$\text{Attenuation factor} = \left(\frac{0.1 - 0.4}{0.1 + 0.4}\right)^2 = \left(\frac{-0.3}{0.5}\right)^2 = (-0.6)^2 = 0.36$$

The between-class separation shrinks to 36% of the original. Starting from $\mu^2 = 4$ (using $\mu = 2$), after aggregation: $\mathbb{E}[\|\bar{\mathbf{h}}_+ - \bar{\mathbf{h}}_-\|^2] \approx 0.36 \times 4 = 1.44$.
With extreme heterophily ($p = 0.01$, $q = 0.99$):

$$\left(\frac{0.01 - 0.99}{0.01 + 0.99}\right)^2 = (-0.98)^2 = 0.9604$$

Wait—this is close to 1! The signal is *almost preserved*? Yes, but the sign has flipped: class $+1$ nodes now have representations that look like class -1 and vice versa. The $(\cdot)^2$ hides the sign flip. The linear factor is $\frac{p-q}{p+q} = -0.98$, meaning the class centroids have essentially swapped positions. A classifier can still separate them, but a naive multi-layer GNN would compound the problem at each layer.
The worst case is $q = p$ (random graph): the attenuation factor is 0, meaning the class signal is completely destroyed.

3.3 Theorem 2: Signal Preservation Under Cost-Sensitive Aggregation

Theorem 3.2 (Cost-sensitive aggregation preserves signal direction)**.** *Under the same CSBM, suppose an aggregation scheme weights edge (i, j) by w_{ij}, with:*

- $\mathbb{E}[w_{ij} \mid y_i = y_j] = w_+$ *(average weight on same-class edges).*
- $\mathbb{E}[w_{ij} \mid y_i \neq y_j] = w_-$ *(average weight on different-class edges).*
- $w_+ > w_- \geq 0$ *(same-class edges get higher average weight).*

Then:

$$\mathbb{E}[\bar{\mathbf{h}}_+^{(1)} - \bar{\mathbf{h}}_-^{(1)}] = \frac{p\,w_+ - q\,w_-}{p\,w_+ + q\,w_-}(\boldsymbol{\mu}_+ - \boldsymbol{\mu}_-) + O(1/n) \qquad \text{(Thm. 2)}$$

The scaling factor is positive—preserving the original class direction—if and only if $w_+/w_- > q/p$.

What it says in plain English: If you can assign higher weights to same-class edges and lower weights to different-class edges, you can control whether the class-mean direction is preserved or reversed after aggregation. The critical condition is that your weight ratio (w_+/w_-) must exceed the graph's heterophily ratio (q/p) to keep the class centroids on the correct side.

Symbol breakdown:

- w_+: the expected weight assigned to same-class edges. In CSNA, this corresponds to the average concordance score s_{ij} for same-class pairs.
- w_-: the expected weight assigned to different-class edges.
- $p\,w_+$: the "effective contribution" of same-class edges (edge probability $\times$ weight).
- $q\,w_-$: the "effective contribution" of different-class edges.
- The formula replaces $p \to p\,w_+$ and $q \to q\,w_-$ in Theorem 1's formula. Cost-sensitive weighting effectively "corrects" the edge probabilities by reweighting them.

The condition $w_+/w_- > q/p$—why it is intuitive:

The Key Condition: $w_+/w_- > q/p$

The graph's heterophily is captured by q/p: this is how much more likely a different-class edge is compared to a same-class edge. For example, if $q/p = 4$, there are 4 times as many cross-class edges as same-class edges.
Your weighting scheme's discrimination is captured by w_+/w_-: how much more weight you assign to same-class edges compared to different-class edges.
The condition says: **your weighting must discriminate edges at least as strongly as the graph's heterophily confuses them**, in order to preserve the correct class direction.
If $q/p = 4$, you need $w_+/w_- > 4$—you must assign same-class edges more than 4 times the weight of different-class edges. The cross-class edges outnumber same-class edges 4-to-1, so your weights must compensate by favoring same-class edges by more than 4-to-1.
When the condition holds, the scaling factor is positive: the class centroids stay on the correct side. When it fails ($w_+/w_- \leq q/p$), the signal direction reverses, just as in unweighted mean aggregation.

Numerical Example

With $p = 0.1$, $q = 0.4$ (so $q/p = 4$):
Mean aggregation ($w_+ = w_- = 1$):

$$\text{Factor} = \left(\frac{0.1 \cdot 1 - 0.4 \cdot 1}{0.1 \cdot 1 + 0.4 \cdot 1}\right)^2 = \left(\frac{-0.3}{0.5}\right)^2 = 0.36$$

Cost-sensitive with $w_+ = 5, w_- = 1$ (ratio $w_+/w_- = 5 > 4 = q/p$):

$$\text{Factor} = \left(\frac{0.1 \cdot 5 - 0.4 \cdot 1}{0.1 \cdot 5 + 0.4 \cdot 1}\right)^2 = \left(\frac{0.5 - 0.4}{0.5 + 0.4}\right)^2 = \left(\frac{0.1}{0.9}\right)^2 \approx 0.012$$

Wait—0.012 is *less* than 0.36! How is this an improvement?
The key is the sign: with mean aggregation, $\frac{p-q}{p+q} = -0.6$ (the signal *flips*). With cost-sensitive weighting, $\frac{pw_+ - qw_-}{pw_+ + qw_-} = +0.11$ (the signal stays in the *correct direction*, though it is small).
For classification, the direction matters: a flipped signal at -0.6 is actually usable (a classifier can learn the flip), but the multi-layer problem is that repeated sign flips across layers cause oscillation. Cost-sensitive weighting avoids this by keeping the signal in the correct direction.
Better cost-sensitive weighting with $w_+ = 10, w_- = 1$:

$$\text{Factor} = \left(\frac{1.0 - 0.4}{1.0 + 0.4}\right)^2 = \left(\frac{0.6}{1.4}\right)^2 \approx 0.184$$

Now the signal retains 18.4% of its original strength in the correct direction, compared to 36% in the wrong direction under mean aggregation. Stronger discrimination ($w_+/w_- = 10$) recovers more of the signal.
Perfect oracle with $w_+ = 1, w_- = 0$ (ignore all cross-class edges):

$$\text{Factor} = \left(\frac{0.1 \cdot 1 - 0.4 \cdot 0}{0.1 \cdot 1 + 0.4 \cdot 0}\right)^2 = \left(\frac{0.1}{0.1}\right)^2 = 1.0$$

Perfect preservation! If you could perfectly identify and ignore all cross-class edges, aggregation would not degrade the signal at all.

3.4 Connecting the Theory to CSNA

How does this apply to CSNA specifically?

Theorem 2 applies to CSNA's concordant channel with:

- $w_{ij} = s_{ij}$ (the concordance score).
- $w_+ = \mathbb{E}[s_{ij} \mid y_i = y_j]$ (average concordance on same-class edges).
- $w_- = \mathbb{E}[s_{ij} \mid y_i \neq y_j]$ (average concordance on different-class edges).

For the theorem's sign-preservation condition to hold, CSNA's learned cost function must assign concordance scores such that $w_+/w_- > q/p$. The **calibration regularizer** (Eq. 9) encourages this by penalizing high costs on same-class edges, which pushes s_{ij} higher for same-class pairs and thus increases w_+/w_-.

Important caveat: Theorem 2 is *conditional*—it says what happens *if* the cost function achieves good separation ($w_+/w_- > q/p$), but it does not guarantee that CSNA *will* learn such separation. Also note that the theorem bounds the class-mean difference, not classification accuracy: a sign-reversed but well-separated representation can still be corrected by a downstream linear classifier. Whether CSNA can actually learn a good cost function depends on the data: on adversarial-heterophily datasets (Texas, Wisconsin, Cornell), the paper shows that CSNA does learn good separation (high AUC for edge-type classification); on informative-heterophily datasets (Chameleon, Squirrel), it does not.

4 Summary of Notation

For quick reference, here is a table of all major symbols used in the paper:

Symbol	Type / Dimensions	Meaning
$\mathcal{G} = (V, E)$	Graph	Undirected graph with nodes V and edges E
$n = \|V\|$	Scalar (integer)	Number of nodes
d	Scalar (integer)	Dimension of node features
d'	Scalar (integer)	Dimension of hidden representations
C	Scalar (integer)	Number of classes
$\mathbf{x}_i \in \mathbb{R}^d$	Vector	Raw feature vector of node i
$\mathbf{X} \in \mathbb{R}^{n \times d}$	Matrix	Feature matrix (row $i = \mathbf{x}_i$)
$y_i \in \{1, \ldots, C\}$	Scalar (integer)	Class label of node i
$\mathcal{H}$	Scalar $\in [0, 1]$	Edge homophily ratio
$\mathcal{N}(i)$	Set of nodes	Neighborhood of i (including i)
$\mathbf{h}_i^{(\ell)} \in \mathbb{R}^d$	Vector	Node i's representation at layer ℓ
$\mathbf{W}^{(\ell)} \in \mathbb{R}^{d' \times d}$	Matrix	Learnable weight matrix at layer ℓ
α_{ij}	Scalar	Aggregation weight for edge (i, j)
$\sigma(\cdot)$	Function	Nonlinear activation (ReLU or sigmoid)
$\mathbf{W}_g \in \mathbb{R}^{d' \times d}$	Matrix	Learned projection for cost computation
g_{ij}	Scalar ≥ 0	Observed divergence (L2 distance in projection)
τ	Scalar > 0	Temperature for sigmoid concordance
s_{ij}	Scalar $\in (0, 1)$	Concordance score
$\tilde{s}_{ij}$	Scalar $\in (0, 1)$	Softmax-normalized concordance weight
$\tilde{d}_{ij}$	Scalar $\in (0, 1)$	Softmax-normalized discordance weight
$\mathbf{W}_{\text{con}}, \mathbf{W}_{\text{dis}} \in \mathbb{R}^{d' \times d}$	Matrices	Channel-specific weight matrices
$\mathbf{W}_{\text{self}} \in \mathbb{R}^{d' \times d}$	Matrix	Ego (self) channel weight matrix
γ_k	Scalar $\in [0, 1]$	Gate weight for channel k ($\sum_k \gamma_k = 1$)
$\mathbf{W}_\gamma \in \mathbb{R}^{3 \times 3d'}$	Matrix	Gate weight matrix
h_{ij}	Scalar ≥ 0	Learned cost component (extended variant)
$\mathbf{a} \in \mathbb{R}^{2d'}$	Vector	Learnable parameter for h_{ij}
f_{ij}	Scalar ≥ 0	Total edge cost ($g_{ij} + h_{ij}$, extended variant)
$\mathcal{L}_{\text{cal}}$	Scalar	Calibration regularization loss
λ_{cal}	Scalar	Regularization weight (fixed at 0.1)
p	Scalar $\in [0, 1]$	CSBM intra-class edge probability
q	Scalar $\in [0, 1]$	CSBM inter-class edge probability
μ	Scalar > 0	CSBM feature signal strength
w_+	Scalar ≥ 0	Expected weight on same-class edges
w_-	Scalar ≥ 0	Expected weight on different-class edges